Advanced JS

Orientation Session : [0]

→ (Basics) only فقط JS اتعودنا نعرف قشور ال -

- What we miss :
  * pros & Cons of every solution
  * Before & After learning DS & Algorithms ⟷ How did you solve it? How much it got improved?

- How to write prompt? Prompt Engineering, Temperature Tokens, ...

- Integrate with OpenAI, Gemini

- You already know the Base of Javascript and you ~~get~~ produce an output, but if you got more Deep, you would get a better o/p with higher performance and better security.

- if you created an object, and don't need it anymore, you'll leave it? that will fill the memory and as a result the performance will go down ↓.

→ Garbage Collector, Hidden Class, String pool

→ Keys order inside the object, methods creation in objects?

→ Security? token in Local, forge?

get token & verify it

null ↓ undefined ↓ "" ↓

→ hole in arrays ↔ how does it affect performance? ....

→ Sometimes you use it in code, but you don't know that it can make a problem after that.

→ Design Patterns, Solid Principles - SOLID

→ Companies look for Devs Engineers have very good knowledge of Basics + Prompt Engineering

↓ How it work?

↓ Multiple Solutions → Best one?

JS → BE, FE, Ai → Tensorflow, React Native --

Output → which o/p is better?

Difference

Mindset Difference?

→ Why the Problem happened and solve it from its roots not ~~eta~~ only solve it.

→ Tracing Code → ✓ Why that problem happened?

not How it will be solved?

→ That's the difference between a

Coder | Developer (Engineer)

not always workarounds ↔ why the char is lower case?

→ So, I have to know how to get/catch the problem from its root

and know when I can say that to solve that issue → That will cost me.

or only make a ~~hot~~ hot fix now, wht It will do?

production issue < hot fix? / from it root?

→ get Back to the Business! Based on my situation

→ Even if you are a Junior, You are assigned to do that!

↳ I've done 1,2,3 -- bec. it has a low cost

So everyone will get impressed! Huraay!!

2nd Part

if I am ~~holg~~ holdig my bag with all my luggage, and I am searchig for a certain thing

Anagram // name .... // mean

not best Solution

```
function is Anagram (a,b) {
  // name === ['n','a','m','e'] => ['a','e','m','n'] => aemn
  // mean === ['m','e','a','n'] => ['a','e','m','n'] => aemn
  return a.split('').sort().join('') ==
         == b.split("").sort().join("")
```

Sort nlogn

DSA ~ 1 month

fast comparison

1 Normal for loop

```
const arr = [1, 2, 3, 4, 5]

for (let i=0; i< arr.length; i++) {
    console.log(i, arr[i]);
}
```

→

0,	1
1,	2
2,	3
3,	4
4,	5

?

→ You control start, condition, incl dec.
→ Best if you ~ need index

2 for ... of

```
for (let item of arr) {
    console.log(item)
}
```

?

→ This directly gives you the value.
→ No indexes involved automatically
→ Best if you only need values.

~

mean

Basic Approach (using object as frequency map)

```
function isAnagram(a, b) {
    if (a.length !== b.length) return false;
        const freq = {};

    // Count chars from a
      for (const char of a) {
          freq[char] = (freq[char] || 0) + 1;
    }

    for (let char of t) {
        if (!freq[char]) return false;
          freq[char]--;
    }
    or: مش موجود أو خلص خلاص
    return true;
}
```

freq = {
a: 1
e: 1
m: 1
n: 1
}

O(n)

You should approach methods other than Built-in methods. O(1) → constant space

here we are looping on all elements, we increment the desired freq. first then check the index of each char directly

We can also use Map

You can search for it

→ we can use an array of size 26 to count the freq of chars (assuming only lowercase letters)

[0, 0, ---] ⇒ Size 26

1- name
a: [1, 0, 0, 0, 1, ----], b: [1, 0, 0, 0, 1, ---]

9 = 0 + 0 ?
in the array ?

Another example

target = 9 , arr = [2, 3, 5, 7]

$O(n^2)$ ⟹ Brute force approach X XX

Optimized approach $O(n)$ ✓ ✓ ✓ ✓

```
function hasPairsWithSum(arr, target) {
    Const map = {}
    for ( let i=0; i<arr.length; i++) {
        Const diff = target - arr[i];
        if(map[diff] !== undefined)
        {
            return map[diff];
        }

        map[arr[i]] = i;
}
```

Mindset

Target : Why we write al for what ?

ليه و عشان ايه

→ Error handling Skill Dv !

→ Is JS blocking or non Blocking ?

↳ Blocking by default ↔ one callStack, one main Thread

→ executes code line by line

But it also can do non-blocking things.

Using ⇒ setTimeout, fetch, promises, async/await event listeners.

Why didn't SetTimeout block?

→ browsers/Web APIs handle it outside the main JS thread

→ Then callback fn goes to the queue
↳ console.log("B") -- for example

→ Event loop pushes it later to Call Stack.

So JavaScript runtime environments support non-blocking async operations.

timers, network, DOM events → help prepare things in background all & notify JS later.

tick ⟹ one cycle of the Event loop